Quantized Human Activity Recognition on an ESP32

Taehee Han · AIoT course project · University of Helsinki
Repository: https://github.com/Rororo06/AIoT

1. Prototype

1.1 Problem and objective

Among older adults, falls are the leading cause of injury-related hospitalisation. What determines the damage is usually not the fall. It is how long the person lies there before anyone notices. A wearable built for this has to decide *locally*, because a device that waits for a network round trip fails in exactly the situations where coverage is worst. Streaming raw 6-axis inertial data all day is also a privacy problem, and it drains the battery.

So I built a wearable-style prototype. It classifies **SITTING / WALKING / FALLING** from an MPU6050, entirely on an ESP32, and I applied **post-training full-integer (int8) quantization** to push it toward the Pareto frontier of accuracy, latency and memory.

I kept the guiding question narrow on purpose:

> *For a 6-axis activity recogniser on an ESP32, what does int8 quantization actually buy, once latency and RAM are measured on the device rather than inferred from the model file size?*

The framing mattered more than I expected. From the host, quantization looks like a 17 % saving on the file I deployed. On the ESP32 the same change is a 4.4x speed-up and a 2.4x reduction in RAM.

1.2 Development process and iterations

Work ran in five stages. Each one ended with a measurement instead of an opinion.

Stage 1, data. I chose SisFall because it is the only widely used public set that carries both the daily activities and the falls I needed for three classes, recorded from the same waist-mounted device. Its institutional download link is dead. The repository therefore fetches a CC BY 4.0 mirror and verifies the archive's SHA-256, which keeps the pipeline reproducible for anyone else.

Stage 2, baseline model. I trained a small CNN at three widths: 811, 2 643 and 9 379 parameters. The point was to be able to trace the trade-off as a curve later, rather than argue it from one point.

Stage 3, quantization. Every width went through float32 and int8 TFLite conversion, and I evaluated the converted artifacts with the TFLite interpreter. That way the accuracy I report belongs to the file that ships, not to the Keras model it came from.

Stage 4, firmware. The model went into an Arduino sketch on TFLite Micro, together with a self test built out of held-out windows.

Stage 5, device measurement. I put all six candidates into a single firmware image and benchmarked them on the simulated ESP32 in one run.

Measurement caught two design errors along the way. Sections 2.4 and 3.4 describe them, because they taught me more than the model did.

1.3 Design decisions

decision	justification
ESP32 over Arduino Uno / Pico	320 KB RAM and 240 MHz make TFLite Micro comfortable; the second core later turned out to be essential
MPU6050, I^2C at 400 kHz	6 axes in one 14-byte burst read (~0.4 ms); the same modality as the training data
±16 g and ±2000 °/s	matches the SisFall sensor ranges, so no value in the training set is unrepresentable by the deployed sensor. Falls routinely exceed 8 g, which is why the third SisFall accelerometer (MMA8451Q, ±8 g) was discarded
50 Hz sampling	the literature reports 50 Hz as sufficient for fall detection, and it is a clean 4x decimation of SisFall's 200 Hz
128-sample window (2.56 s)	contains more than two gait cycles *and* the complete free-fall → impact → rest sequence
stride 64 (50 % overlap)	a decision every 1.28 s, which becomes the hard real-time budget in §3.5
three classes only	the goal is a demonstrable edge trade-off study, not a general HAR system

1.4 Abandoned approaches

Random window splitting. Split windows at random and accuracy goes up, but only because overlapping windows from one trial end up on both sides of the split. I split by *subject* instead: 26 train, 5 validate, 7 test, nobody appearing twice. The 0.93 macro-F1 I report is a good deal lower than a random split would give me. It is also the only version I would trust.

Keras `Conv1D`. This was the natural way to write the model, and I dropped it after looking at the converted graph. The TFLite converter turns every `Conv1D` into an `EXPAND_DIMS`, `CONV_2D`, `RESHAPE` triplet. That left the deployed model with 16 operators and pulled two extra kernels into the firmware. Writing the identical computation with an explicit `(time, 1, channel)` layout and `Conv2D` gave this instead:

topology	int8 bytes	operators	kernels to register
Conv1D (abandoned)	7 376	16	CONV_2D, EXPAND_DIMS, FULLY_CONNECTED, MAX_POOL_2D, MEAN, RESHAPE, SOFTMAX
Conv2D (deployed)	**4 976**	**8**	CONV_2D, FULLY_CONNECTED, MAX_POOL_2D, MEAN, SOFTMAX

`python tools/compare_topologies.py` reproduces the comparison.

BatchNorm went too. It folds away cleanly, but leaving it out keeps the float and int8 graphs directly comparable and holds the operator set down.

`AllOpsResolver` became `MicroMutableOpResolver<5>`. Only the five kernels the model really uses are registered now.

---

2. Sensing pipeline

2.1 Hardware, signals and communication

```
MPU6050 ──I²C 400 kHz──> ESP32 ──> ring buffer ──> TFLM int8 CNN ──> serial + 3 LEDs
 ±16 g, ±2000 °/s          core 0        128×6           core 1
 DLPF 21 Hz               50 Hz        2.56 s          every 1.28 s
```

- **Reading mode:** a polled burst read of registers `0x3B` to `0x48`, giving six 16-bit two's-complement values per sample. I do not use the interrupt pin. The sample clock comes from the RTOS scheduler.
- **Sampling frequency:** 50 Hz, so one sample every 20 ms.
- **Communication:** I²C at 400 kHz on GPIO 21 and 22, address `0x68`. One 14-byte burst costs about 0.4 ms, roughly 2 % of the sample period. There is no wireless link at all. The classification never leaves the chip, which is the privacy and energy argument for edge AI in its most concrete form.
- **Output:** the class, all three probabilities and the inference time go out on serial. One LED per class lights up, but only when the smoothed decision is confident.

2.2 Anti-aliasing: a hardware decision, not a software one

At 50 Hz the Nyquist limit is 25 Hz, and the transients from a body hitting the floor carry plenty of energy above that. I therefore set the MPU6050's digital low-pass filter to **21 Hz**, under Nyquist, so the aliasing is killed inside the sensor and never reaches the MCU. The training data gets the same treatment. SisFall's 200 Hz recordings are decimated to 50 Hz through a zero-phase FIR filter, and a unit test checks the result: an 80 Hz component has to fall below 5 % while a 5 Hz component survives.

2.3 From counts to tensor

1. Raw int16 register values.
2. Physical units: `g = counts / 2048` and `°/s = counts / 16.384`, the MPU6050 sensitivities for the ranges I configured.
3. A ring buffer of 128 samples by 6 channels, kept in physical units.
4. Per-channel standardisation, using statistics from the **training split only**, compiled into the firmware as 12 constants.
5. Quantisation to int8, with scale and zero-point read from the model at runtime. They come out as `scale = 0.40271258` and `zero_point = -16`.

Steps 2 to 5 run on the device for live data and for the built-in self test alike. That is why the self test stores its held-out windows as raw register values: it has to exercise the whole chain, not just the interpreter.

2.4 Scheduling: the error that measurement caught

One int8 inference takes about 94 ms. That is 4.7 sample periods. My first firmware ran the inference inline in `loop()`, which meant the 50 Hz stream came to a halt every single time the model ran. The firmware's own counters put a number on it:

scheduling	worst sample lateness	samples skipped
inline in `loop()` (abandoned)	89 094 µs	24 over 7 windows (~4 per window)
sampler task pinned to core 0 (shipped)	**0 µs**	**0**

The version I shipped moves acquisition into a FreeRTOS task on core 0 at priority 3, driven by `vTaskDelayUntil` so the 20 ms period does not drift. Once a window is complete the producer snapshots it under a mutex and gives a binary semaphore. The Arduino loop on core 1 wakes up, copies the snapshot and runs the interpreter. I kept the broken version buildable behind `-DUSE_SAMPLING_TASK=0`, so the comparison above is reproducible.

If the project taught me one thing, it is this. **On an edge device the model's latency is a scheduling constraint, not a quality-of-service number you can trade against accuracy whenever you feel like it.**

2.5 Energy budget

I had no power meter. What follows is worked out from datasheet currents and the duty cycle I did measure. It is an estimate and should be read as one.

component	condition	current	duty	contribution
MPU6050	accel + gyro active	3.9 mA	100 %	3.9 mA
ESP32	active, radios off	~40 mA	100 %	40 mA
ESP32	inference above idle	~15 mA	7.4 %	1.1 mA

At 3.3 V that comes to roughly 149 mW. Notice what dominates: the MCU simply being awake, not the inference. Three consequences follow, and they shaped the design.

- **Wi-Fi and Bluetooth stay off.** One Wi-Fi transmit burst pulls around 240 mA, more than seconds of local inference cost. Classifying on the device and sending only events is the energy-optimal architecture, not a compromise.
- **Inference is duty cycled** to one window in every 64 samples. With int8 that is 7.4 % of the CPU. The same network in float32 wants 32.3 %. Quantization is an energy technique here as much as a memory one.
- **The gyroscope dominates the sensor budget.** It is the obvious next target. Going accelerometer-only would cut sensor current by roughly four times.

3. On-device AI

3.1 Model

Input `(128, 1, 6)`, then Conv2D(8, 5x1, ReLU), MaxPool 2x1, Conv2D(16, 3x1, ReLU), MaxPool 2x1, GlobalAveragePooling, Dense(8, ReLU), Dense(3, softmax).

That is 811 parameters. Using global average pooling instead of flatten and dense is the single choice that keeps the count two orders of magnitude below a typical HAR CNN.

Training used Adam at an initial learning rate of 1e-3, batch size 64, up to 60 epochs. Early stopping watched validation loss with patience 10, and `ReduceLROnPlateau` halved the rate after 5 flat epochs. Class weights handled the residual imbalance. Seed 1337 throughout.

3.2 Data

split	subjects	windows	SITTING	WALKING	FALLING
train	26	5 639	2 264	2 250	1 125
val	5	1 377	483	596	298
test	7	2 899	588	1 936	375

No window was labelled by hand. Rules produce all of them, which makes them reproducible. WALKING comes from `D01`, `D02`, `D05` and `D06` with 2 s trimmed off each end. SITTING is the longest quasi-stationary stretch inside the sit and stand trials `D07` to `D10`. FALLING is one window, centred on the impact peak of `F01` to `F15`.

WALKING is capped in train and validation so it cannot swamp the loss. The **test split keeps its natural distribution** untouched, which is why I report macro-F1 and per-class recall rather than leaning on accuracy.

3.3 Optimization technique: full-integer int8 quantization

Exact converter settings:

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset  # 300 TRAIN windows
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.int8
converter.inference_output_type = tf.int8
```

- Weights are **per-channel int8**. Activations are **per-tensor int8**.
- Calibration uses **300 class-stratified windows drawn from the training split only**. Never validation, never test.
- The **input tensor is int8** as well, so the ESP32 quantises the standardised window itself and no float kernels get linked into the firmware at all.

3.4 Results before and after optimization

Accuracy comes from the 2 899-window subject-wise test split. Latency and arena were measured on the ESP32 by the firmware itself.

variant	precision	.tflite	ESP32 arena	ESP32 latency	macro-F1	FALLING recall
small	float32	6 632 B	8 504 B	413.3 ms	0.9327	0.984
small	**int8**	**5 520 B**	**3 484 B**	**94.4 ms**	**0.9312**	**0.981**
medium	float32	13 984 B	13 624 B	1 139.4 ms	0.9292	0.981
medium	int8	8 240 B	5 020 B	221.5 ms	0.9308	0.984
large	float32	40 904 B	25 912 B	3 534.4 ms	0.9318	0.987
large	int8	16 680 B	8 668 B	618.2 ms	0.9327	0.987

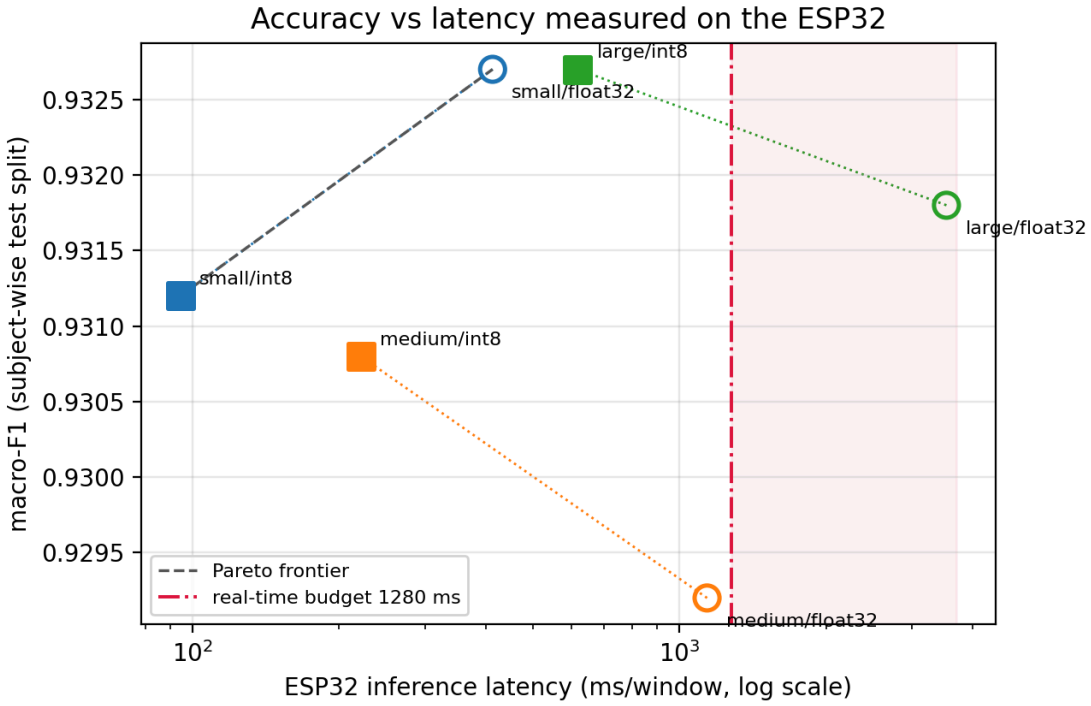
Effect of quantization alone:

variant	size	arena	latency	macro-F1
small	-16.8 %	**2.44x smaller**	**4.38x faster**	-0.0015
medium	-41.1 %	**2.71x smaller**	**5.14x faster**	+0.0016
large	-59.2 %	**2.99x smaller**	**5.72x faster**	+0.0009

Building the same sketch two ways gives a whole-firmware footprint of 382 756 B flash and 40 460 B static RAM for int8, against 383 880 B and 45 580 B for float32.

Accuracy does not really move. For medium and large it even comes out slightly *higher* after quantization. I would not read that as an improvement. It looks like int8 noise acting as a weak regulariser at this model scale.

3.5 Pareto frontier



A decision falls due every 64 samples, so the system lives under a hard budget of **1 280 ms per inference**. Three things follow from that line.

The real-time constraint kills candidates, and quantization brings them back. `large/float32` needs 3 534 ms, which is 276 % of the decision interval. It is not slow, it is infeasible. `medium/float32` at 1 139 ms eats 89 % of the budget and leaves nothing for anything else. Quantize both and they return: `large/int8` sits at 48 % duty, `medium/int8` at 17 %. This is not a marginal gain. It decides whether the system runs at all.

The host-side frontier lies. Rank the six candidates by `.tflite` size and they look almost interchangeable, with the int8 saving on my deployed model coming to a modest 17 %. Rank them by measured latency and arena and the same change is worth 4.4x and 2.4x. Drawing the frontier from file sizes is the intuitive move, and it would have led me to the wrong answer.

What I deployed. Two points on the measured frontier are non-dominated: `small/int8` at 94 ms, 3 484 B and macro-F1 0.9312, and `small/float32` at 413 ms, 8 504 B and 0.9327. `large/int8` ties the best macro-F1 of 0.9327, but it wants 6.5 times the latency and 2.5 times the RAM, so it is dominated. I deployed **`small/int8`**. Trading 0.0015 macro-F1 for 4.4x less latency and 2.4x less RAM is the right call for something running off a battery, and it leaves 92.6 % of the CPU free for the sampling task and whatever comes next.

Capacity buys almost nothing here. Eleven times the parameters shifts macro-F1 by 0.0015. That tells me the ceiling sits in the data and the label rules, not in the network. Spending the freed budget on more classes, or on better labels, would pay off far more than a bigger model.

3.6 Per-class behaviour of the deployed model

This is `small/int8` over the 2 899 test windows. Rows are true classes, columns are predictions.

	pred SITTING	pred WALKING	pred FALLING	precision	recall	F1
SITTING (588)	**574**	14	0	0.768	0.976	0.860
WALKING (1 936)	173	**1 760**	3	0.988	0.909	0.947
FALLING (375)	0	7	**368**	0.992	0.981	0.987

Where the errors sit matters more than how many there are. **7 falls out of 375 are missed.** Not one of them is confused with SITTING. The safety-critical direction is the strong one, which is what I care about: a fall almost never gets quietly ignored. Falls also cost just 3 false alarms across 1 936 walking windows, so the alarm stays credible.

The genuine weakness runs the other way: 173 WALKING windows, 8.9 % of them, get called SITTING, and that is what drags SITTING precision down to 0.768. I read this as a labelling artefact rather than a model failure. The stair-walking trials `D05` and `D06` contain real pauses at the landings, and a 2.56 s window that lands on a pause looks like sitting even to me, reading the traces by hand. A longer window would fix it and blunt fall detection at the same time, so I resolved the trade-off in favour of the safety-critical class.

3.7 Correctness evidence

- The firmware self test takes 9 held-out test windows, decodes them from raw MPU6050 register values and pushes them through the full pipeline. All 9 come back correct, and that holds for each of the six candidates.
- Three Wokwi automation scenarios replay held-out windows *through the simulated sensor over I^2C*, which is why nothing needs to be hard coded. SITTING landed at p = 0.844, WALKING at p = 0.996, FALLING at p = 0.996, all on the expected class.
- 14 unit tests cover unit conversion, decimation, anti-aliasing, all three label rules, split disjointness and the firmware code generation.

---

4. Self-reflection

4.1 Limitations

- **The simulator cannot validate my sensing assumption.** Wokwi replays recorded windows faithfully enough, but it has nothing to say about a sensor mounted elsewhere, a rotated device or a real wearer. All I can claim is that the system works on waist-mounted data in the orientation it was recorded in.
- **Domain gap.** The training data comes off an ADXL345 and ITG3200 pair. The deployment target is an MPU6050. I matched ranges and sensitivities on purpose, but noise characteristics and filter behaviour still differ, and simulation cannot surface that difference.
- **Simulated timing.** Every latency figure is simulator-measured. I trust the int8 against float32 comparison, since both ran on the same simulated core, but the absolute milliseconds are not a hardware guarantee. The reference TFLite Micro kernels are also unoptimised for Xtensa. ESP-NN would probably cut both numbers a long way without moving their ratio.
- **Estimated energy.** I could not measure power at all.
- **Falls are simulated.** Volunteers fell onto mattresses, and the elderly participants barely performed falls at all, so real FALLING recall for the group this is meant to protect is probably below the 0.981 I report.
- **WALKING recall of 0.905 is my weakest number**, mostly lost to SITTING. The stationary phases inside the stair-walking trials are genuinely ambiguous at a window length of 2.56 s.

4.2 A course concept applied: the Pareto frontier

The AI-on-Edge chapter treats edge deployment as multi-objective optimisation. There is no single best model, only a set of non-dominated ones. I took that literally, and it shaped the whole project. Rather than tune one network, I trained three widths in both precisions to get six candidates, then traced the frontier from measurements instead of intuition.

That exposed something the concept predicts but a single-model experiment would never show me. The shape of the frontier **depends on which axis you measure**. Along the file-size axis my six points nearly collapse into one. Along the device-latency axis they span a factor of 37, and two of them sit outside the feasible region altogether. The practical lesson is uncomfortable: a Pareto analysis is only as good as the realism of its objectives, and a proxy metric will hand you a confident wrong answer without complaining.

4.3 How my understanding of AIoT changed

I came in believing on-device AI is basically a compression problem. Shrink the network until it fits, then deploy. Two measurements took that apart.

The first was `Conv1D`. One line in Keras turned into three operators on the device and dragged two extra kernels into my firmware. The model I *express* and the model I *deploy* are not the same artifact, and only the second one costs anything.

The second was worse, because it was quiet. A 94 ms inference was blocking a 20 ms sampling loop. Model latency is not a quality metric I get to trade against accuracy at my leisure. It is a scheduling constraint, and it was corrupting the input to the very model I was busy optimising. Nothing crashed. Nothing logged an error. The windows were just slightly wrong.

My mental model shifted from "*compress a model to fit a device*" to "*co-design a sensing pipeline, a schedule and a model against one shared budget.*" Quantization mattered here less because it shrank a file and more because it turned 32 % CPU duty into 7 %, and that is what made a clean dual-core schedule affordable at all. The quantities that matter on an edge device are not the ones a training notebook prints. That is why every number in this report that could be measured on the device was measured there.

4.4 What I would do next

Ranked by expected value, not by how easy they are.

1. **Fix the labels before touching the model.** 173 of my 197 test errors are WALKING windows landing on stair pauses. A segmentation-aware label rule would buy more than eleven times the parameters did.
2. **Validate on real hardware.** Two assumptions sit outside what a simulator can test: the domain gap between the ADXL345 and ITG3200 pair and an MPU6050, and the sensitivity to how the device is mounted.
3. **Link ESP-NN.** The reference Xtensa kernels leave a large factor unclaimed. Optimised ones would push the 94 ms down further and widen the feasible region of the frontier.
4. **Duty cycle the MCU.** It is the always-awake ESP32 that dominates my energy estimate, not the inference. Light sleep between samples is worth more than any further compression of the model.

Appendix: reproduction

```
pip install -r requirements.txt
python tools/download_dataset.py && python -m training.prepare_data
python -m training.train && python -m training.quantize && python -m training.evaluate
python -m training.export_firmware small && python -m training.export_model_zoo
python -m training.export_wokwi && python -m pytest -q
python tools/measure_firmware.py          # needs arduino-cli + esp32 core
python tools/run_simulation.py            # needs WOKWI_CLI_TOKEN
```

Artifacts: `results/metrics.csv`, `results/ondevice_metrics.json`, `results/firmware_sizes.json`, `results/quantization_effect.md`, `results/pareto_*.png`, `results/wokwi_*.log`.